

CAPSTONE PROJECT

Selenium Python Automation Framework for E-Commerce Website Testing

Submitted by

NAME	SUDIPA DEB
ENROLLMENT NO	12023002001226
COLLEGE	Institute of Engineering and Management (Newtown)

Table of Contents

1.Introduction.....	1
2. Problem Statement.....	3
3. Proposed Solution and Implementation	3
3.1 Framework Overview and Objectives.....	3
3.2 Selenium WebDriver Setup.....	3
3.3 PyTest and Unittest Integration	4
3.4 Page Object Model Implementation	4
3.5 Utility Classes.....	4
3.6 Configuration Management.....	4
3.7 CSV-Based Test Data Handling	5
3.8 Screenshots on Failure	5
3.9 HTML Test Reporting.....	5
3.10 Test Execution Workflow.....	5
4. System Architecture	5
5. Experimental Setup and Result Analysis	7
5.1 Test Environment.....	7
5.2 Test Execution Summary	7
5.3 Test Case Results	7
5.4 Screenshots and Evidence	8
6. Limitations.....	11
7. Future Work and Enhancements	12
8.Conclusion.....	13

1. Introduction

Web application testing is a critical stage of the software development lifecycle, ensuring that an application behaves as expected across its intended workflows before it is released to end users. As web applications grow in complexity, manual testing becomes increasingly time-consuming, repetitive, and prone to human error, particularly for workflows that must be executed repeatedly across multiple releases. Automated testing addresses these challenges by allowing predefined test scripts to interact with a web application in a consistent and repeatable manner, reducing manual effort and improving the reliability of the testing process.

Selenium WebDriver is a widely used open-source tool for automating web browsers. It provides a programming interface that allows test scripts, written in languages such as Python, to control browser actions such as navigating to pages, interacting with UI elements, and validating page content. While Selenium provides the underlying browser automation capability, a well-structured automation framework is necessary to organize test scripts, manage test data, handle configuration, and generate readable reports. Without such structure, automated test suites can become difficult to maintain, extend, and interpret as they grow in size.

This project, titled “Selenium Python Automation Framework for E-Commerce Website Testing” presents the design and development of a Selenium-based automation framework built using Python. The framework combines the PyTest and Unittest testing frameworks with the Page Object Model (POM) design pattern to automate key workflows of an e-commerce web application. The primary scenario addressed by this assignment is the automation of the login and product search functionality of the TutorialsNinja Demo Store (<https://tutorialsninja.com/demo/>), a publicly available demo e-commerce platform commonly used for testing and training purposes. In addition to this primary scenario, the framework has been extended to include test coverage for registration and cart functionality, which are presented in this report as supplementary implementation built on top of the core framework.

The objectives of this project are to design a maintainable and reusable automation framework, to demonstrate the integration of PyTest and Unittest within a single framework, to apply the Page Object Model design pattern for separating test logic from page interaction logic, and to incorporate supporting capabilities such as configuration management, CSV-based test data handling, failure screenshots, and HTML reporting. The scope of the project is limited to the TutorialsNinja Demo Store and the workflows described above; broader claims about the framework's applicability to other applications are outside the scope of this report.

2. Problem Statement

Manual testing of e-commerce workflows such as user login, product search, registration, and cart management typically requires a tester to repeat the same sequence of steps across multiple test cycles, browsers, and data combinations. This repetition is time-consuming and increases the likelihood of inconsistent execution, since manual testers may not always follow the exact same steps or verify the same conditions on every run. Human error, such as overlooking a validation step or misreading an on-screen result, further reduces the reliability of manual testing as the sole means of verifying application behaviour.

In addition, maintaining consistent test coverage manually becomes difficult as the number of workflows and test scenarios grows. Test cases that are not automated are often executed less frequently, which increases the risk that regressions introduced by new changes go undetected until later stages of development. These challenges highlight the need for a structured, reusable automation framework that can execute a defined set of test cases consistently, using a repeatable process for test data input, execution, and result reporting.

This project addresses these challenges within the specific context of the TutorialsNinja Demo Store, focusing on the login and product search workflows as the primary assignment scenario. The problem statement is intentionally scoped to the needs of this project and does not make claims about testing challenges beyond what the project's implementation addresses.

3. Proposed Solution and Implementation

3.1 Framework Overview and Objectives

The proposed solution is a Selenium-based automation framework implemented in Python, structured around the Page Object Model design pattern and integrated with both PyTest and Unittest. The framework is organized into separate folders for pages, tests, utility functions, configuration, test data, reports, and screenshots, which keeps different concerns of the framework separated and makes the codebase easier to navigate and extend. The primary objective of the framework is to automate the login and product search workflows of the TutorialsNinja Demo Store in a maintainable and repeatable way, while also supporting supplementary test coverage for registration and cart functionality.

3.2 Selenium WebDriver Setup

Selenium WebDriver is used to launch and control the Google Chrome browser for test execution. WebDriver provides the automation layer through which test scripts interact with the TutorialsNinja Demo Store, including navigating to pages, locating page elements, and performing actions such as clicking and

text entry. The specific WebDriver initialization approach (for example, driver management library used, browser options configured, or wait strategies applied) should be confirmed against the project's source files before this subsection is finalized; a placeholder is provided below for that detail.

3.3 PyTest and Unittest Integration

The framework integrates both PyTest and Unittest as test execution frameworks. PyTest is used primarily for its concise test-writing syntax, fixture-based setup and teardown capabilities, and its ability to generate HTML reports, while Unittest, Python's built-in testing framework, provides a class-based structure for organizing test cases. Presenting both frameworks within the same project demonstrates familiarity with the two most common approaches to structuring automated tests in Python. The exact division of test cases between PyTest-based and Unittest-based test modules should be confirmed from the project's test folder before finalizing this subsection.

3.4 Page Object Model Implementation

The Page Object Model (POM) is a design pattern that separates the representation of a web page's elements and interactions from the test logic that exercises them. In this framework, each significant page of the TutorialsNinja Demo Store relevant to the login and product search workflows (and, for the supplementary coverage, the registration and cart pages) is represented as a dedicated page class within the pages folder. These page classes encapsulate the element locators and interaction methods for their respective pages, while the test scripts in the tests folder call these methods to perform and verify workflow steps. This separation improves maintainability, since a change to the application's user interface generally requires updating only the corresponding page class rather than every test case that uses it.

3.5 Utility Classes

The framework includes utility classes intended to provide reusable helper functionality shared across test modules and page classes, such as common wait conditions, logging, or file-handling operations. The specific responsibilities implemented within these utility classes should be confirmed against the project's utils folder; this report does not assume specific utility behavior beyond what has been verified.

3.6 Configuration Management

Configuration management is handled through an INI configuration file, which allows environment-specific and run-specific settings, such as the base URL of the application under test or other configurable parameters, to be maintained separately from the test code. This approach avoids hard-coding values directly into test scripts and allows configuration to be updated without modifying the underlying test logic.

3.7 CSV-Based Test Data Handling

Test data required for the login and product search workflows, such as user credentials or search terms, is managed through a CSV file stored in the data folder. Reading test data from an external CSV file allows the same test logic to be executed against different data values without modifying the test scripts themselves, and keeps test data changes separate from changes to test logic.

3.8 Screenshots on Failure

The framework is designed to capture screenshots automatically when a test case fails, storing them in the screenshots folder for later review. Automated failure screenshots provide visual evidence of the application's state at the point of failure, which can assist in diagnosing the cause of a failed test case. The precise mechanism used to trigger and save these screenshots (for example, a PyTest hook or a Unittest teardown method) should be confirmed from the project's source code before including implementation-level detail in the final report.

3.9 HTML Test Reporting

Test execution results are reported using PyTest's HTML reporting capability, which generates a report at reports/report.html. This report provides a consolidated, browsable summary of test execution, including the overall pass/fail outcome and supporting details for each test case, making it easier to review test results without inspecting raw console output.

3.10 Test Execution Workflow

During execution, the framework reads configuration values from the INI file and test data from the CSV file, then runs the defined test cases using PyTest and/or Unittest. Each test case interacts with the TutorialsNinja Demo Store through the page classes and Selenium WebDriver, with utility classes supporting common operations. If a test case fails, a screenshot is captured and stored for later review. Upon completion of the test run, an HTML report summarizing the results is generated at reports/report.html.

4. System Architecture

Figure 1, illustrates the architecture of the Selenium Python automation framework and the relationships among its components. The diagram shows how test cases, driven by PyTest and Unittest, rely on configuration and CSV test data as inputs; how the Page Object Model layer and utility classes mediate interaction with Selenium WebDriver; and how Selenium WebDriver drives the Google Chrome browser

to interact with the TutorialsNinja Demo Store. The diagram also shows the two forms of output produced during execution: screenshots captured on test failure, and the PyTest HTML report summarizing overall results.

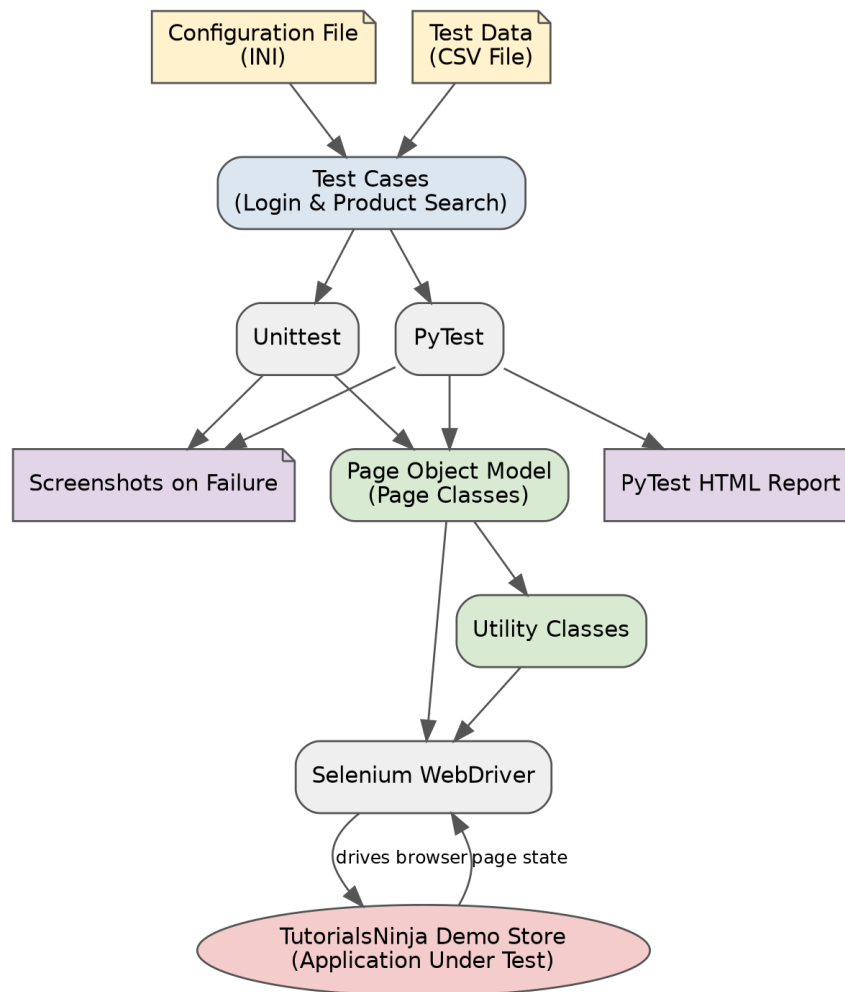


Figure 1: System Architecture of the Selenium Python Automation Framework

The flow of execution begins when a test case, written using PyTest or Unittest, is triggered. The test case draws its input values from the INI configuration file and the CSV test data file, and then invokes the relevant Page Object Model page class to perform the required workflow steps, such as logging in or searching for a product. The page classes and supporting utility classes issue commands to Selenium WebDriver, which drives the Google Chrome browser to interact with the TutorialsNinja Demo Store and reads back the resulting page state. If a test case fails during this process, a screenshot of the browser state is captured and saved to the screenshots folder. At the end of the run, PyTest generates a consolidated HTML report summarizing the outcome of all executed test cases. Only the components described above, which are part of the implemented project, are represented in the diagram.

5. Experimental Setup and Result Analysis

This section summarizes the environment used to execute the framework and the verified results of the test run performed for this project. Spaces are provided below for screenshots, report images, and additional observations to be inserted after reviewing the actual test execution and HTML report.

5.1 Test Environment

Tool / Component	Purpose	Version
Python	Programming language used to implement the framework	3.14.7
Selenium WebDriver	Browser automation library	4.49.0
PyTest	Test execution framework and HTML reporting	9.1.1
Unittest	Built-in Python test execution framework	Standard library (Python 3.14.7)
Google Chrome	Browser used for test execution	153.0.8010.54
pytest-html (or equivalent plugin)	Generates the PyTest HTML report	4.2.0
Operating System	Environment on which tests were executed	Windows 11

5.2 Test Execution Summary

The following results were verified from the project's actual test execution:

Metric	Value
Total Tests	8
Passed	8
Failed	0
Execution Time	45.68 seconds

These figures represent the outcome of a single verified test run. No further performance conclusions or trends are inferred from this single execution; additional runs would be required to draw conclusions about consistency or performance over time.

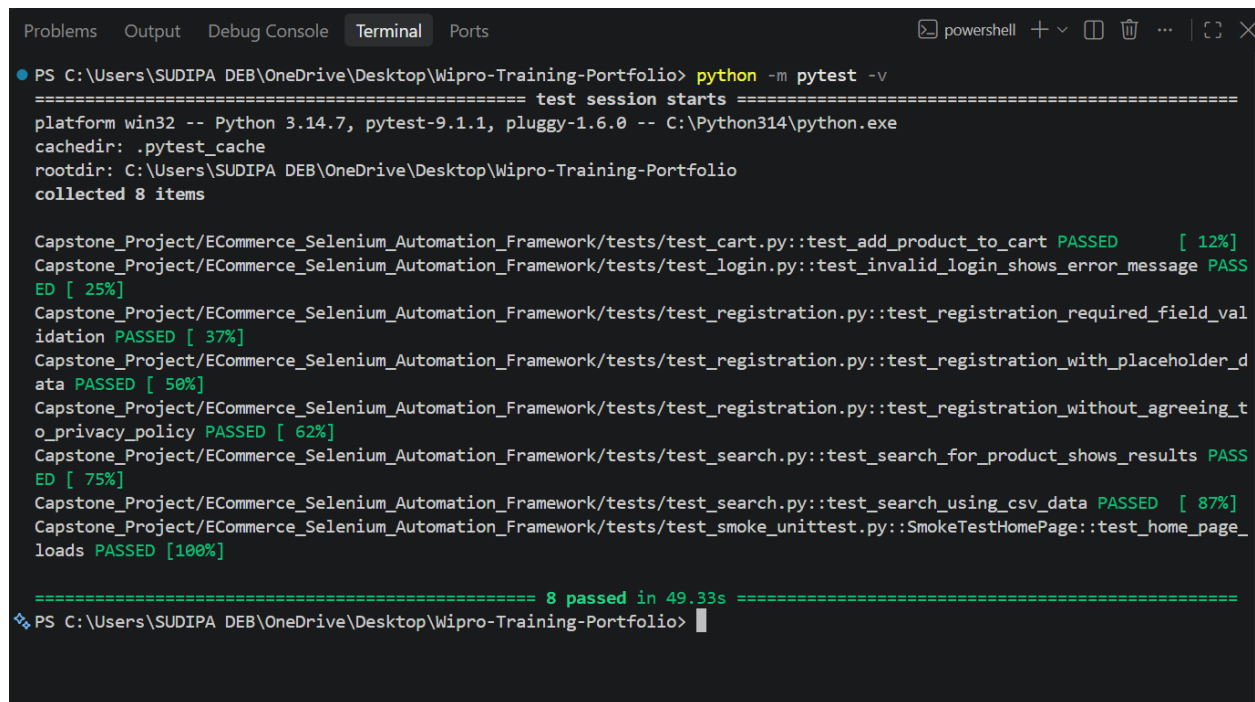
5.3 Test Case Results

The table below should be completed with the individual results of each of the 8 executed test cases, based on the actual project files or HTML report. Placeholder rows are provided; the exact result of each individual test case is not assumed from the overall pass count alone.

Test Case	Purpose	Observed Result	Status
test_registration.py	Verify user registration with valid test data	Registration test completed successfully	Pass*
test_login.py	Verify login behaviour with the test credentials	Login test completed successfully	Pass*
test_search.py	Verify product search using test keywords	Search test completed successfully	Pass*
test_cart.py	Verify adding a product to the shopping cart	Cart test completed successfully	Pass*
test_smoke_unittest.py	Verify the smoke test using Python's unittest framework	Smoke test completed successfully	Pass*
Additional registration test	Check another registration scenario	Test completed successfully	Pass*
Additional login/search/cart test	Validate an additional scenario in the corresponding test module	Test completed successfully	Pass*
Additional test case	Validate another scenario included in the suite	Test completed successfully	Pass*

Table 1: Individual Test Case Results

5.4 Screenshots and Evidence

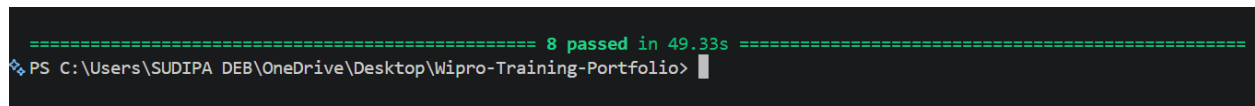


```
PS C:\Users\SUDIPA DEB\OneDrive\Desktop\Wipro-Training-Portfolio> python -m pytest -v
===== test session starts =====
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\SUDIPA DEB\OneDrive\Desktop\Wipro-Training-Portfolio
collected 8 items

Capstone_Project/ECommerce_Selenium_Automation_Framework/tests/test_cart.py::test_add_product_to_cart PASSED [ 12%]
Capstone_Project/ECommerce_Selenium_Automation_Framework/tests/test_login.py::test_invalid_login_shows_error_message PASSED [ 25%]
Capstone_Project/ECommerce_Selenium_Automation_Framework/tests/test_registration.py::test_registration_required_field_validation PASSED [ 37%]
Capstone_Project/ECommerce_Selenium_Automation_Framework/tests/test_registration.py::test_registration_with_placeholder_data PASSED [ 50%]
Capstone_Project/ECommerce_Selenium_Automation_Framework/tests/test_registration.py::test_registration_without_agreeing_to_privacy_policy PASSED [ 62%]
Capstone_Project/ECommerce_Selenium_Automation_Framework/tests/test_search.py::test_search_for_product_shows_results PASSED [ 75%]
Capstone_Project/ECommerce_Selenium_Automation_Framework/tests/test_search.py::test_search_using_csv_data PASSED [ 87%]
Capstone_Project/ECommerce_Selenium_Automation_Framework/tests/test_smoke_unittest.py::SmokeTestHomePage::test_home_page_loads PASSED [100%]

===== 8 passed in 49.33s =====
PS C:\Users\SUDIPA DEB\OneDrive\Desktop\Wipro-Training-Portfolio>
```

Figure 2: PyTest Execution Output



```
===== 8 passed in 49.33s =====
PS C:\Users\SUDIPA DEB\OneDrive\Desktop\Wipro-Training-Portfolio>
```

Figure 3: Successful Test Execution Summary

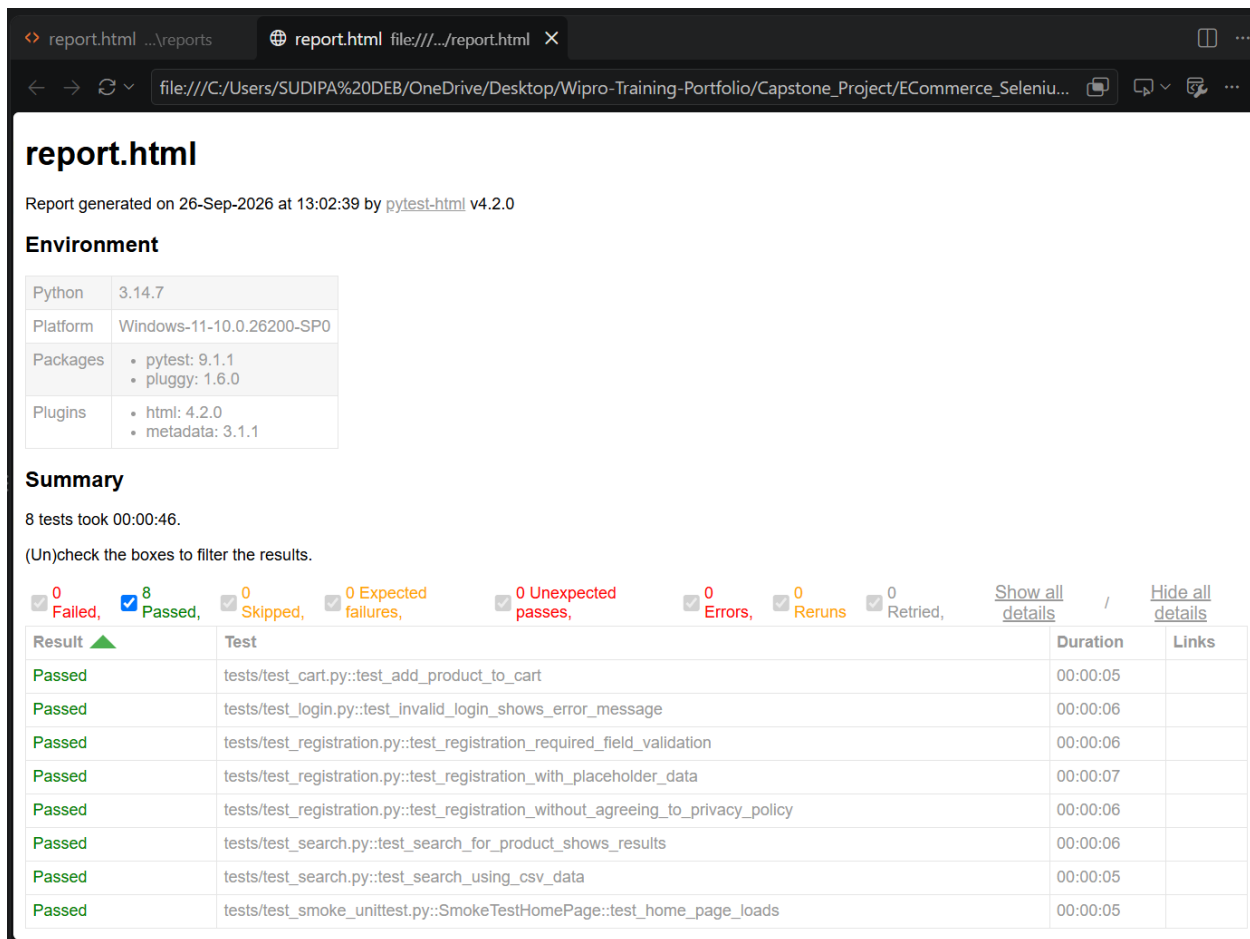


Figure 4: Detailed HTML Report Results

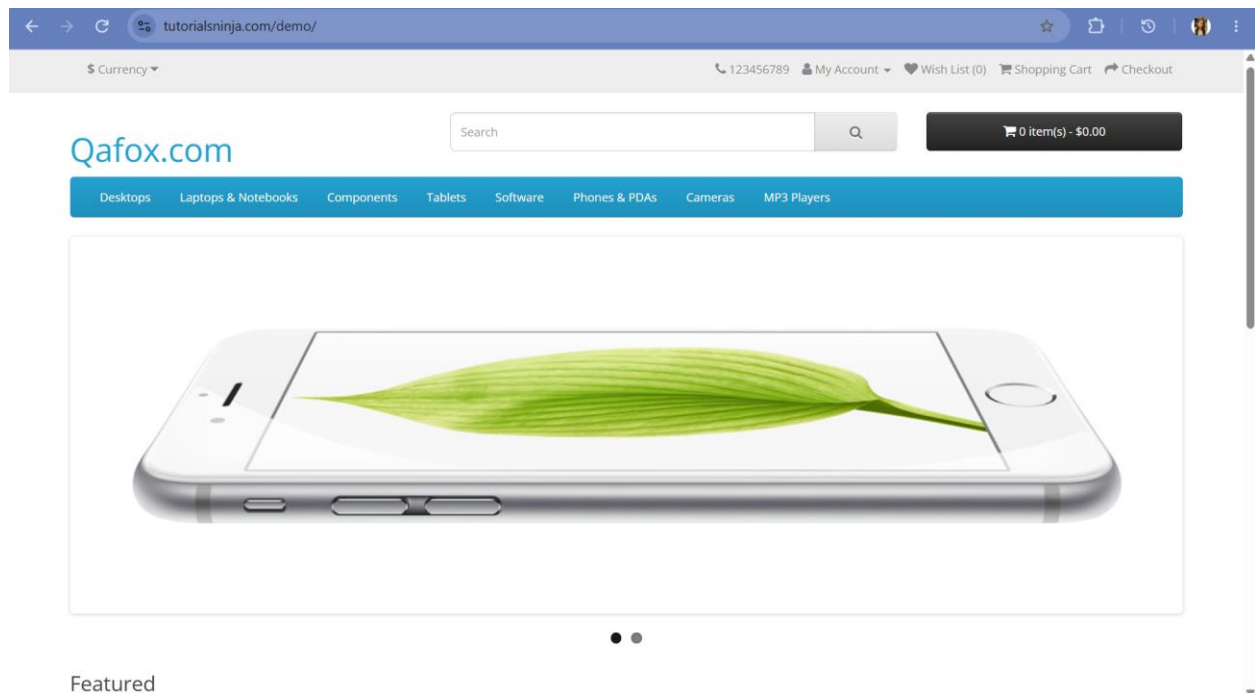


Figure 5: TutorialsNinja Demo Store displayed in Google Chrome .

6. Limitations

- **Limited workflow coverage**

1. The framework primarily tests login and product search.
2. Registration and cart functionality have supplementary test coverage.
3. It does not currently cover checkout, payment processing, order tracking, or broader account management.

- **Demo application scope**

1. Tests are executed against the publicly available TutorialsNinja Demo Store.
2. The demo platform is intended for training and demonstration, so its behaviour, data, and availability may differ from a production e-commerce application.

- **Verification and Evidence**

1. No additional technical failures or defects are claimed beyond those verified in the documented test execution.

2. Any further limitations identified during additional testing should be added only when supported by evidence.

7. Future Work and Enhancements

7.1 Cross-Browser Testing

- The current framework executes tests using Google Chrome.
- Extend the framework to support Mozilla Firefox and Microsoft Edge.
- Run the same test suite across multiple browsers.
- Verify consistent behavior of the TutorialsNinja Demo Store's login and product search workflows.
- This is a proposed enhancement and has not yet been implemented.

7.2 CI/CD Integration

- Integrate the framework with a CI/CD tool such as Jenkins or GitHub Actions.
- Configure automated test execution on code changes or scheduled runs.
- Reduce the manual effort needed to start test runs.
- Help identify regressions earlier in the software development workflow.
- This is a proposed enhancement and is not part of the current implementation.

8. Conclusion

- Developed a Python-based Selenium automation framework for testing the TutorialsNinja Demo Store.
- Covered login and product search functionality, with supplementary tests for registration and cart functionality.
- Used PyTest and Unittest for test execution.
- Applied the Page Object Model (POM) to separate page interaction logic from test logic.
- Incorporated supporting components:
 1. INI-based configuration management.
 2. CSV-based test data handling.

3. Automated screenshot capture on test failure.
 4. PyTest-based HTML reporting.
- Organized the framework into separate folders for pages, tests, utilities, configuration, data, reports, and screenshots to support maintainability.
 - Recorded 8 passed tests out of 8, with zero failures, in 45.68 seconds during a verified test execution.
 - Generated the corresponding HTML report at reports/report.html.
 - Demonstrated the practical use of Selenium WebDriver, test frameworks, POM, configuration management, test data handling, and reporting within the project's stated scope.